

UNIVERSITÀ DI BOLOGNA

Corso di Laurea in Ingegneria e Scienze Informatiche

Predictive Auto-Scaling in Kubernetes

Corso di **Virtualizzazione e Integrazione di Sistemi**

Responsabile del corso: **Vittorio Ghini**

Studente

Francesco Meloni

Matricola: 00011144018

Anno Accademico

2025/2026

Indice

1	Introduzione	4
1.1	Contesto	4
1.2	Il problema dello scaling	5
1.3	Il progetto	5
1.4	Struttura della relazione	5
1.5	Obiettivi	6
2	Tecnologie Utilizzate	7
2.1	Docker e containerizzazione	7
2.1.1	Immagini e Dockerfile	7
2.2	Kubernetes	8
2.2.1	Astrazioni principali	8
2.2.2	Architettura del cluster	9
2.2.3	Il ciclo di controllo	10
2.2.4	HorizontalPodAutoscaler	10
2.3	Minikube	11
2.4	NGINX Ingress	11
2.4.1	Come funziona	11
2.5	Prometheus	12
2.5.1	Modello pull	12
2.5.2	Modello dati	12
2.5.3	PromQL	13
2.5.4	Service Discovery in Kubernetes	13
2.6	Python e Flask	13
2.6.1	WSGI — Web Server Gateway Interface	14
2.6.2	Flask	14
2.6.3	prometheus_client	15
2.7	Locust	15
2.7.1	Componenti principali	15
2.7.2	Architettura distribuita	16
2.8	KEDA	16

2.8.1	Il trigger di Prometheus	16
2.8.2	Architettura interna	17
2.8.3	Flusso di scaling dettagliato	18
2.8.4	ScaledObject	18
2.9	Prophet	19
2.9.1	Fitting e overfitting	19
2.9.2	Trend $g(t)$	20
2.9.3	Stagionalità $s(t)$	20
2.9.4	Festività $h(t)$	21
2.9.5	Termine di errore ε_t	21
2.9.6	Intervalli di incertezza	21
3	Architettura del Sistema	22
3.1	Visione d'insieme	22
3.2	Componenti applicativi	22
3.3	Ciclo predittivo	23
3.4	Struttura del progetto	23
3.5	Infrastruttura Kubernetes	23
3.5.1	Networking e Ingress	23
3.5.2	Prometheus	24
3.5.3	KEDA	24
4	Implementazione	25
4.1	Microservizi Flask	25
4.2	Forecast Service	26
4.2.1	Raccolta dati da Prometheus	26
4.2.2	Gestione della finestra storica	26
4.2.3	Modello Prophet	27
4.2.4	Generazione della previsione	27
4.2.5	Loop principale	28
4.3	Configurazione KEDA	28
4.4	Generatore di Carico	29
4.4.1	Motivazione del profilo sinusoidale	29
4.4.2	Implementazione con Locust	29
5	Risultati Sperimentali	32
5.1	Configurazione della simulazione	32
5.2	Profilo di traffico generato	32
5.3	Metriche predette dal Forecast Service	33
5.3.1	Periodo di warm-up	33

5.3.2	Fase predittiva	33
5.4	Comportamento dello scaling	33
5.4.1	Numero di repliche nel tempo	33
5.4.2	Evidenza dello scaling	34
5.5	Interfacce utente dei servizi	34
6	Conclusioni	35
6.1	Sintesi dei risultati	35
6.2	Limiti del sistema	35
6.3	Possibili evoluzioni	36

Capitolo 1

Introduzione

1.1 Contesto

Il paradigma del *cloud computing* ha trasformato radicalmente il modo in cui le applicazioni vengono progettate, distribuite e gestite. Il **cloud computing** è un modello di erogazione di servizi informatici — elaborazione, storage, rete — tramite Internet, secondo un paradigma *on-demand* e a pagamento proporzionale all'uso. Il National Institute of Standards and Technology (NIST) definisce il cloud computing come un modello che abilita un accesso di rete ubiquo, conveniente e *on-demand* a un pool condiviso di risorse computazionali configurabili (reti, server, storage, applicazioni e servizi) che possono essere rapidamente provisionate e rilasciate con un minimo sforzo gestionale [1].

I tre principali modelli di servizio sono:

- **IaaS** (Infrastructure as a Service): noleggio di infrastruttura virtualizzata (VM, storage, rete);
- **PaaS** (Platform as a Service): ambienti di esecuzione e strumenti di sviluppo gestiti;
- **SaaS** (Software as a Service): applicazioni complete erogate come servizio (Caso di questo progetto).

Le architetture **cloud-native**, basate su **microservizi** containerizzati e orchestrati da sistemi come Kubernetes [2], permettono di scomporre applicazioni complesse in componenti *indipendenti* e *scalabili*. Questo approccio porta con sé un vantaggio fondamentale: la possibilità di adattare dinamicamente le risorse computazionali al carico di lavoro effettivo, pagando solo per ciò che si utilizza e garantendo al contempo prestazioni adeguate agli utenti.

1.2 Il problema dello scaling

La gestione delle risorse computazionali in un ambiente cloud è un problema di ottimizzazione con due obiettivi contrapposti: ridurre i costi e garantire le prestazioni. L'**auto-scaling** automatizza questo bilanciamento adattando dinamicamente il numero di istanze attive al carico corrente.

L'approccio tradizionale, noto come **reactive auto-scaling**, funziona misurando metriche del sistema (utilizzo CPU, numero di richieste al secondo, latenza) e reagendo quando queste superano una soglia predefinita. Il limite intrinseco di questo approccio è il **ritardo di reazione**: il tempo che intercorre tra il manifestarsi del picco di traffico e l'effettiva disponibilità di nuove istanze può essere nell'ordine dei minuti, durante i quali il servizio opera in condizioni di sovraccarico con conseguente degradazione delle prestazioni per gli utenti finali.

Una soluzione alternativa è il **predictive auto-scaling**: anziché reagire a metriche già misurate, il sistema anticipa le variazioni di carico sulla base di previsioni generate da modelli statistici o di machine learning, allocando le risorse necessarie prima che il picco si manifesti.

1.3 Il progetto

Questo progetto propone e implementa un sistema completo di predictive auto-scaling interamente basato su tecnologie open source, eseguito su un cluster Kubernetes locale gestito da Minikube. Il sistema è composto da due **microservizi applicativi** — un generatore di numeri casuali (*Random Service*) e un calcolatore di hash SHA-256 (*Hash Service*). Il traffico verso questi servizi è generato sinteticamente da un **Load Generator** con profilo sinusoidale a periodo di 24 ore, modellando il tipico *diurnal pattern* del traffico web [3, 4].

Al centro del sistema si trova il **Forecast Service**: un componente che raccoglie continuamente le metriche di traffico da **Prometheus** [5], le usa per addestrare un modello di previsione basato su **Prophet** [6] (la libreria di Meta per le serie temporali) e pubblica la previsione del traffico futuro come nuova metrica Prometheus. **KEDA** [7] (Kubernetes Event-Driven Autoscaler) legge questa previsione e scala i microservizi *prima* che il traffico reale raggiunga il picco, eliminando il ritardo tipico dello scaling reattivo.

1.4 Struttura della relazione

Il capitolo 2 descrive in dettaglio le tecnologie utilizzate, fornendo il necessario background teorico. Il capitolo 3 illustra l'architettura complessiva del sistema e i flussi di

dati tra i componenti. Il capitolo 4 approfondisce le scelte implementative più rilevanti. Il capitolo 5 presenta i risultati sperimentali ottenuti. Il capitolo 6 riassume i risultati e discute le possibili evoluzioni del progetto.

1.5 Obiettivi

1. Progettare e deployare un'architettura a microservizi su Kubernetes con monitoraggio e auto-scaling;
2. Implementare un servizio di previsione del traffico basato su Prophet, integrato con Prometheus per la raccolta e la pubblicazione di metriche;
3. Configurare KEDA per usare le previsioni come trigger di scaling;
4. Simulare il diurnal pattern con un generatore di carico sinusoidale a periodo giornaliero;
5. Valutare quantitativamente il comportamento del sistema.

Capitolo 2

Tecnologie Utilizzate

2.1 Docker e containerizzazione

Docker è una piattaforma open source per la creazione, distribuzione ed esecuzione di applicazioni all'interno di **container** [8]. Un container è un processo isolato che condivide il kernel del sistema operativo host ma con il proprio filesystem, rete e spazio dei processi, tutto questo in un pacchetto isolato e portabile. A differenza delle macchine virtuali, i container non includono un OS completo, risultando ordini di grandezza più leggeri e veloci da avviare.

2.1.1 Immagini e Dockerfile

Un'**immagine** Docker è un template immutabile a strati (*layers*) da cui vengono istanziati i container. Ogni strato rappresenta una modifica rispetto al precedente ed è cachato individualmente, rendendo efficienti le ricostruzioni incrementali. Il **Dockerfile** è il file di configurazione che descrive come costruire l'immagine a partire da un'immagine base e delle operazioni. Un'operazione del Dockerfile (copia file, esecuzione comando) è una modifica all'immagine iniziale, quindi un layer aggiuntivo.

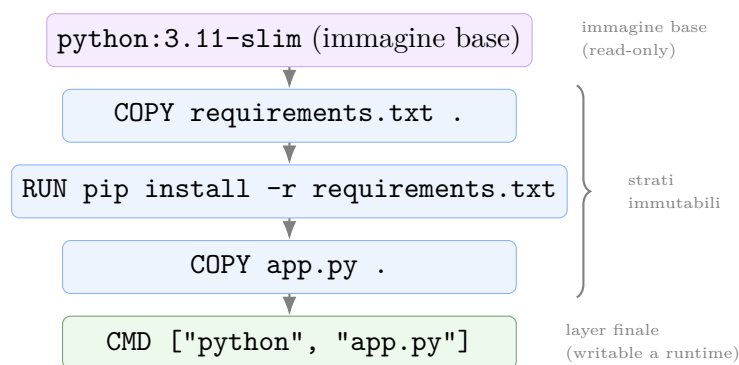


Figura 2.1: Struttura a strati di un'immagine Docker.

2.2 Kubernetes

Kubernetes è il sistema standard de facto per l'orchestrazione di container, donato da Google alla CNCF nel 2014 [2]. Automatizza deployment, scaling e gestione di applicazioni containerizzate in cluster.

2.2.1 Astrazioni principali

Kubernetes organizza le applicazioni attraverso una gerarchia di oggetti:

- **Pod:** unità atomica di esecuzione, contiene uno o più container che condividono rete e storage locali. È effimero: può essere creato, distrutto e ricreato dal sistema in qualsiasi momento;
- **Deployment:** controller che gestisce un insieme replicato di pod, garantendone il numero desiderato (*desired replicas*) e supportando aggiornamenti rolling senza downtime;
- **Service:** astrazione di rete stabile che espone un insieme di pod tramite un indirizzo IP virtuale fisso e bilancia il traffico tra le istanze;
- **Ingress:** regola l'accesso HTTP/HTTPS esterno, fungendo da reverse proxy con routing basato su hostname e path;
- **ConfigMap:** dizionario chiave-valore per configurazione di dati non sensibile (URL di servizi, flag, file di configurazione). I valori vengono iniettati nei pod come variabili d'ambiente, argomenti di comando o file montati su volume;
- **Secret:** analogo al ConfigMap ma per dati sensibili (password, token API, certificati TLS). I valori sono codificati in base64 e l'accesso è controllato tramite RBAC (*Role-Based Access Control*). Possono essere cifrati a riposo con provider KMS (*Key Management Service*).

2.2.2 Architettura del cluster

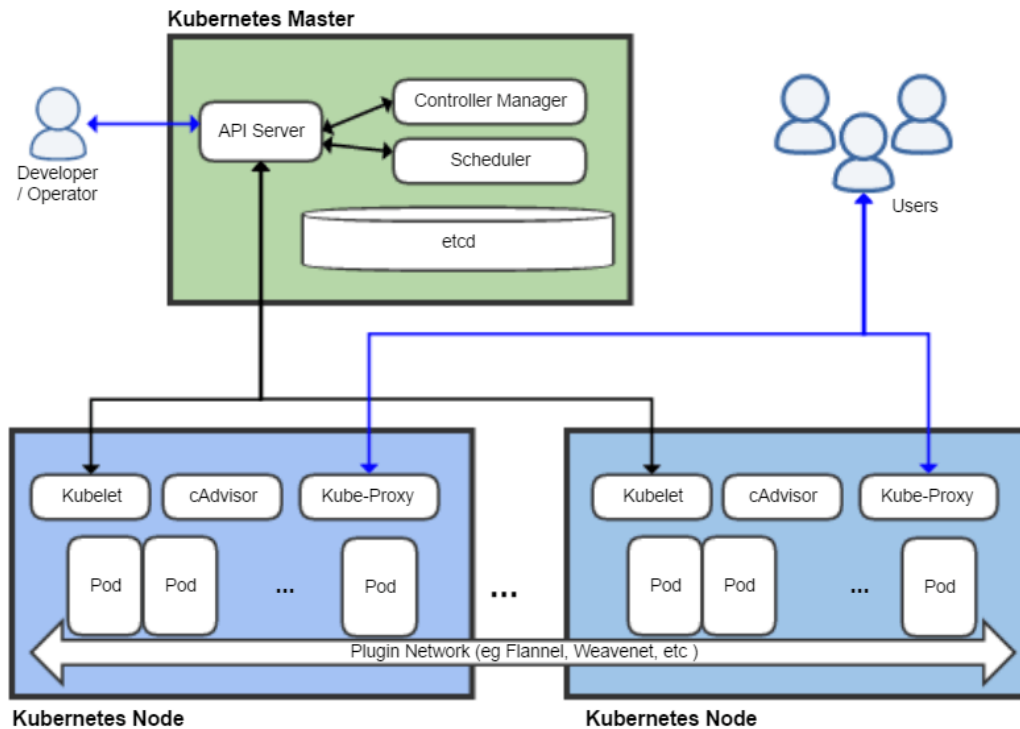


Figura 2.2: Architettura semplificata di un cluster Kubernetes.

Il **control plane** mantiene lo stato desiderato del cluster tramite un insieme di controller che implementano il *control loop*:

- **API Server**: unico punto d'ingresso per tutte le operazioni sul cluster; espone le API REST di Kubernetes;
- **etcd**: database chiave-valore distribuito e fortemente consistente che persiste l'intero stato del cluster;
- **Scheduler**: assegna i pod ai nodi tenendo conto di risorse disponibili, affinità e anti-affinità;
- **Controller Manager**: esegue i controller (Deployment, ReplicaSet, HPA...) che riconciliano lo stato attuale con quello desiderato.

Ogni **worker node** esegue le seguenti componenti:

- **kubelet**: agente principale del nodo. Riceve le specifiche dei pod (PodSpec) dall'API Server e garantisce che i container siano in esecuzione e sani. Gestisce avvio, restart in caso di crash e terminazione dei pod, aggiornandone lo stato nel cluster;

- **kube-proxy**: mantiene le regole di rete (iptables/IPVS) per il bilanciamento del traffico tra i pod di un Service. Ogni volta che un Service viene creato o aggiornato, kube-proxy aggiorna le regole affinché il traffico raggiunga i pod corretti;
- **cAdvisor** (Container Advisor): agente integrato nel kubelet [9] che raccoglie metriche di utilizzo delle risorse per ogni container: CPU, memoria (RSS, cache, working set), I/O di rete e disco. Le espone all'endpoint `/metrics/cadvisor` del kubelet, da cui Prometheus le raccoglie tramite il job `kubernetes-nodes-cadvisor`;
- **Plugin Network (CNI)**: plugin che implementa la rete overlay del cluster secondo la specifica Container Network Interface [10]. Assegna un indirizzo IP univoco a ogni pod e garantisce la comunicazione tra pod su nodi diversi senza NAT. I plugin più diffusi sono **Flannel** (semplice, sviluppo), **Calico** (Network-Policy avanzate, produzione) e **Cilium** (eBPF, alte prestazioni). Minikube usa **kindnet** di default;
- **Pod**: gestiti dal kubelet in base alle PodSpec ricevute dall'API Server. Il kubelet monitora lo stato dei pod tramite *liveness* e *readiness probe* e li riavvia in caso di crash secondo la *restartPolicy*.

2.2.3 Il ciclo di controllo

Il meccanismo fondamentale di Kubernetes è il **control loop** (ciclo di controllo), dove ogni controller:

1. Osserva lo *stato attuale* del cluster (es. 1 replica attiva);
2. Lo confronta con lo *stato desiderato*, definito nelle risorse Kubernetes (es. 3 repliche);
3. Se i due stati differiscono intraprende le azioni necessarie per riconciliarli (avvia 2 nuovi pod).

Questo paradigma, noto come *reconciliation loop*, garantisce che il sistema converga sempre verso lo stato desiderato, anche in presenza di guasti: se un pod si arresta inaspettatamente, ne viene avviato immediatamente uno nuovo.

2.2.4 HorizontalPodAutoscaler

L'**HPA** è il meccanismo nativo di Kubernetes per lo scaling orizzontale automatico. Opera su un ciclo di controllo che, ad intervalli regolari (default 15 s), confronta il

valore corrente di una metrica con il valore desiderato e aggiusta il numero di repliche del Deployment target secondo la formula:

$$\text{replicas}_{\text{desired}} = \left\lceil \text{replicas}_{\text{current}} \cdot \frac{\text{metricValue}_{\text{current}}}{\text{metricValue}_{\text{desired}}} \right\rceil \quad (2.1)$$

L'HPA supporta metriche di risorse (CPU, memoria) nativamente, e metriche custom o esterne tramite adapter dell'*External Metrics API*. KEDA implementa questo adapter, estendendo l'HPA con decine di sorgenti di metriche esterne.

2.3 Minikube

Minikube è uno strumento ufficiale del progetto Kubernetes che permette di eseguire un cluster Kubernetes single-node in locale, ideale per lo sviluppo e il testing [11]. Supporta diversi *driver* di virtualizzazione: Docker, VirtualBox, Hyper-V, KVM. Nel presente progetto viene utilizzato il driver Docker, che fa girare il nodo Kubernetes all'interno di un container Docker, eliminando la necessità di una macchina virtuale separata e rendendo il setup leggero e veloce.

Minikube espone funzionalità aggiuntive tramite **addon**:

- **ingress**: installa il controller NGINX per la gestione degli oggetti Ingress nel cluster;
- **ingress-dns**: configura un resolver DNS locale che risolve automaticamente gli hostname degli Ingress verso l'IP di Minikube, evitando la modifica manuale del file `hosts`.

2.4 NGINX Ingress

NGINX è un web server ad alte prestazioni, ampiamente usato anche come *reverse proxy* e *load balancer*. Il **NGINX Ingress Controller** è un componente Kubernetes che implementa la risorsa Ingress usando NGINX come proxy.

2.4.1 Come funziona

Quando una richiesta HTTP arriva al cluster, NGINX legge l'header `Host` e la inoltra al Service Kubernetes corrispondente, che bilancia il traffico tra i pod disponibili. La configurazione è dichiarativa tramite la risorsa `Ingress` di Kubernetes: ogni regola specifica hostname, path e backend.

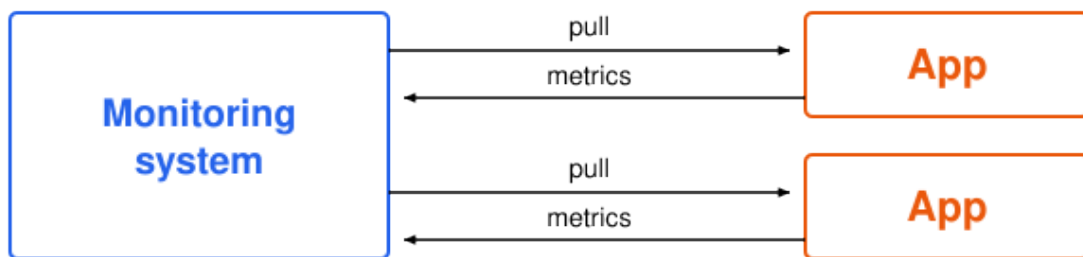
2.5 Prometheus

Prometheus è un sistema di monitoraggio e database di serie temporali open source, sviluppato originariamente da SoundCloud nel 2012 e ora parte della CNCF [5]. È diventato lo standard de facto per il monitoraggio di applicazioni cloud-native grazie alla sua architettura semplice, al modello dati flessibile e al potente linguaggio di query.

2.5.1 Modello pull

A differenza di sistemi push-based (dove i servizi inviano attivamente le metriche a un collettore centrale), Prometheus adotta un **modello pull**: è il server Prometheus stesso che interroga periodicamente gli endpoint `/metrics` di ciascun target configurato. Questo approccio semplifica la configurazione dei servizi monitorati (che devono solo esporre un endpoint HTTP) e permette a Prometheus di rilevare autonomamente i target non raggiungibili.

Pull-based system



Push-based monitoring system

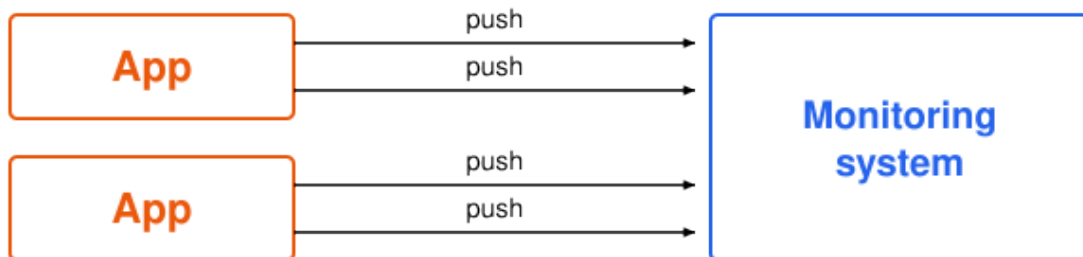


Figura 2.3: Modello pull di Prometheus: il server interroga attivamente i target.

2.5.2 Modello dati

Ogni metrica in Prometheus è una **serie temporale** identificata da:

- un **nome** (es. `random_requests_total`);

- un insieme di **label** (coppie chiave-valore) che permettono di distinguere istanze diverse della stessa metrica (es. `instance`, `job`, `pod`).

Ogni serie temporale è una sequenza di campioni (*timestamp*, *valore_float64*), memorizzati in modo efficiente nel database interno di Prometheus (TSDB).

2.5.3 PromQL

PromQL (*Prometheus Query Language*) è il linguaggio funzionale per interrogare le serie temporali. Le funzioni più rilevanti per il progetto sono:

- `rate(counter[window])`: calcola il tasso di crescita per secondo di un counter nell'intervallo di tempo specificato. È la funzione fondamentale per convertire un counter cumulativo in un rate istantaneo. È robusta al restart dei pod perché lavora sulla variazione del valore;
- `sum(...)`: aggrega più serie temporali sommandone i valori, utile per ottenere il rate totale di più pod dello stesso servizio;
- `avg_over_time(gauge[window])`: calcola la media di un gauge nell'intervallo specificato, usata da KEDA per smussare le fluttuazioni della metrica predetta.

2.5.4 Service Discovery in Kubernetes

Prometheus individua i target da monitorare tramite la sua integrazione con l'API di Kubernetes. I pod che espongono metriche vengono annotati con:

- `prometheus.io/scrape: "true"`: abilita lo scraping del pod;
- `prometheus.io/port`: specifica la porta dell'endpoint `/metrics`;
- `prometheus.io/path`: specifica il path (default `/metrics`).

Prometheus legge queste annotazioni e configura automaticamente lo scraping, senza necessità di aggiornare manualmente i file di configurazione.

2.6 Python e Flask

I microservizi applicativi del progetto sono implementati in **Python** con il micro-framework **Flask** [12]. Python è un linguaggio di programmazione ad alto livello, interpretato e a tipizzazione dinamica, ampiamente usato nello sviluppo web e nell'analisi dei dati. La sua ricca ecosistema di librerie lo rende particolarmente adatto per progetti che combinano servizi web e modelli statistici, come in questo caso.

2.6.1 WSGI — Web Server Gateway Interface

WSGI (PEP 3333) è la specifica standard Python che definisce l'interfaccia tra un **web server** (es. Nginx, Gunicorn) e un'**applicazione web** Python [13]. Prima di WSGI ogni framework usava interfacce proprietarie.

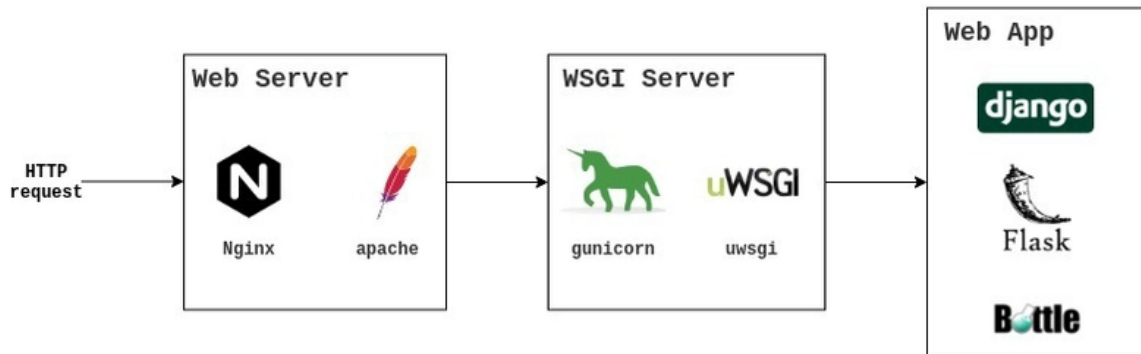


Figura 2.4: Architettura WSGI.

Come mostrato in figura 2.4, l'architettura è composta da tre livelli:

1. **Web Server** (es. Nginx, Apache): gestisce le connessioni HTTP in ingresso, lo static file serving, TLS/SSL e il bilanciamento del carico. Non esegue codice Python direttamente; delega le richieste dinamiche al WSGI Server tramite socket Unix o TCP;
2. **WSGI Server** (es. Gunicorn, uWSGI): esegue il codice Python. Riceve le richieste dal Web Server, costruisce il dizionario `environ` (metodo HTTP, path, header, body) e chiama l'applicazione per ogni richiesta. Gestisce il pool di worker process e la concurrency;
3. **Web Application** (es. Django, Flask, Bottle): il codice applicativo. Un'applicazione WSGI è qualsiasi callable Python con firma `app(environ, start_response)` che restituisce un iterabile di byte come risposta.

Nel presente progetto Flask viene usato con il server di sviluppo integrato (non adatto alla produzione ma sufficiente per la simulazione), che implementa direttamente l'interfaccia WSGI senza un web server intermedio.

2.6.2 Flask

Flask è un micro-framework WSGI minimalista: non impone strutture rigide all'applicazione. Ogni endpoint viene definito con il decorator `@app.route`, che registra la funzione come handler per un path URL. La semplicità di Flask lo rende ideale per microservizi con responsabilità singola e ben definita.

2.6.3 prometheus_client

La libreria **prometheus_client** fornisce i tipi di dati metrici di Prometheus direttamente in Python. I tipi principali utilizzati nel progetto sono:

- **Counter**: valore intero monotonicamente crescente, usato per contare eventi cumulativi (es. numero totale di richieste ricevute). Non può diminuire, ed è per questo robusto al restart del processo;
- **Gauge**: valore numerico che può salire e scendere liberamente, usato per quantità istantanee (es. valore predetto del traffico).

La funzione `generate_latest()` serializza tutte le metriche registrate nel formato di testo atteso da Prometheus, pronto per essere servito all'endpoint `/metrics`.

2.7 Locust

Locust [14] è un framework open source per il load testing scritto interamente in Python. A differenza di strumenti tradizionali basati su thread (es. JMeter), Locust utilizza **coroutine event**: ogni utente virtuale è una greenlet — un thread cooperativo ultra-leggero — che esegue il proprio ciclo di richieste in modo asincrono. Questo permette di simulare migliaia di utenti concorrenti su una singola macchina con un consumo di memoria minimo.

2.7.1 Componenti principali

- **HttpUser**: classe base da estendere per definire il comportamento degli utenti virtuali. Ogni istanza rappresenta un utente che esegue i task decorati con `@task`. Il parametro `wait_time` definisce l'intervallo di attesa tra un task e il successivo: con `between(a, b)` l'attesa è estratta uniformemente da $[a, b]$ secondi, simulando il *think time* di un utente reale;
- **@task**: decorator che marca un metodo come task eseguibile. Locust seleziona casualmente quale task eseguire ad ogni ciclo; il parametro peso (es. `@task(2)`) regola la probabilità di selezione;
- **LoadTestShape**: classe che permette di controllare dinamicamente il numero di utenti virtuali nel tempo. Il metodo `tick()` viene chiamato ogni secondo e restituisce la coppia (`user_count`, `spawn_rate`): il numero totale di utenti desiderati e la velocità con cui aggiungerli o rimuoverli. Restituire `None` termina il test.

2.7.2 Architettura distribuita

Locust supporta una modalità **master/worker** per distribuire il carico su più macchine: un nodo master coordina il test e aggrega le statistiche, mentre i nodi worker generano effettivamente il traffico. Nel presente progetto viene usata la modalità single-process (headless), sufficiente per il carico simulato.

2.8 KEDA

KEDA (Kubernetes Event-Driven Autoscaler) è un componente open source che estende le capacità di scaling di Kubernetes permettendo di scalare Deployment in risposta a eventi provenienti da sorgenti esterne [7]. Nato da una collaborazione tra Microsoft e Red Hat, è oggi un progetto CNCF di livello *graduated*.

2.8.1 Il trigger di Prometheus

In KEDA, un **trigger** è il componente logico incaricato di interfacciarsi con una fonte di eventi esterna (come code di messaggi, database o sistemi di monitoraggio) per stabilire se e come scalare un'applicazione. Il **trigger di Prometheus** permette specificamente di basare le decisioni di scaling orizzontale sull'esito di interrogazioni fatte sui dati metrici raccolti dal server Prometheus.

Operativamente, configurando questo trigger, KEDA esegue periodicamente una determinata query PromQL e ne valuta il risultato numerico per determinare lo stato del sistema. Un trigger è considerato **attivo** quando la query restituisce un valore **strettamente maggiore di zero**: questa condizione segnala che la sorgente di eventi è attiva e che il Workload deve essere scalato. Il trigger è invece **inattivo** in due casi:

- il valore restituito è esattamente zero: la sorgente esiste ma non produce eventi;
- la query non restituisce alcun risultato: la serie temporale è assente in Prometheus (il servizio non è ancora avviato, non ha ancora prodotto dati, oppure lo scraping non è ancora avvenuto).

2.8.2 Architettura interna

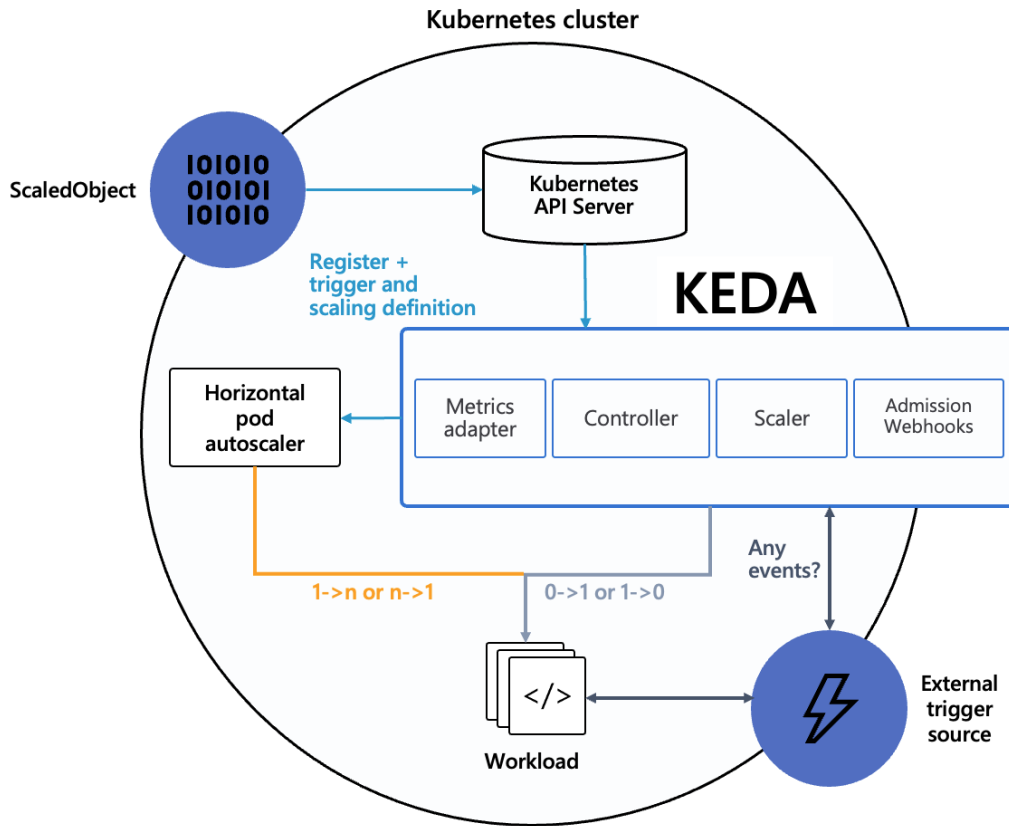


Figura 2.5: Architettura KEDA: dal `ScaledObject` al `Workload` [7].

Come illustrato in figura 2.5, KEDA è composto da quattro componenti interni e si interfaccia con le risorse Kubernetes standard.

- **Controller:** il componente centrale di KEDA. Osserva le risorse `ScaledObject` registrate nell'API Server, interroga periodicamente le sorgenti esterne (*External trigger source*, es. Prometheus) per verificare la presenza di eventi, e decide se e come modificare il numero di repliche del `Workload`;
- **Metrics Adapter:** implementa l'*External Metrics API* di Kubernetes. Espone le metriche delle sorgenti esterne all'HPA nativo, che le usa per calcolare il numero di repliche desiderato;
- **Scaler:** plugin specifico per ogni sorgente supportata (Prometheus, Kafka, RabbitMQ, ...). Si occupa di interrogare la sorgente esterna e determinare se il trigger è attivo (*Any events?*);

- **Admission Webhooks:** validano le risorse `ScaledObject` al momento della creazione, garantendo che la configurazione sia corretta prima di essere applicata al cluster.

Il flusso mostrato in figura prevede due percorsi di scaling distinti:

- **0→1 o 1→0** (freccia grigia): quando un trigger passa da inattivo ad attivo (o viceversa), KEDA gestisce direttamente il Workload, portandolo da zero a almeno una replica o viceversa. Questo percorso bypassa l'HPA e permette lo *scale-to-zero*, funzionalità assente nell'HPA nativo;
- **1→n o n→1** (freccia arancione): quando il Workload è già attivo (≥ 1 replica), KEDA delega lo scaling all'**Horizontal Pod Autoscaler**, che calcola il numero di repliche desiderato in base alla metrica fornita dal Metrics Adapter e aggiorna il Deployment.

2.8.3 Flusso di scaling dettagliato

1. L'utente crea un `ScaledObject` che descrive target e trigger;
2. Il **keda-operator** rileva il `ScaledObject` e interroga periodicamente le sorgenti dei trigger;
3. Se almeno un trigger è **attivo** (valore > 0), il keda-operator crea un **HPA** puntato al Deployment target;
4. L'HPA interroga il **keda-metrics-apiserver** tramite l'External Metrics API per il valore corrente della metrica;
5. L'HPA calcola le repliche secondo l'equazione (2.2) e aggiorna il Deployment;
6. Il **Deployment controller** crea o rimuove i pod necessari;
7. Se tutti i trigger diventano inattivi e `minReplicaCount = 0`, il keda-operator bypassa l'HPA e scala il Deployment a zero repliche (*scale-to-zero*), funzionalità assente nell'HPA nativo.

2.8.4 ScaledObject

La risorsa custom `ScaledObject` (CRD — Custom Resource Definition) è il meccanismo principale di configurazione di KEDA. Definisce:

- Il **target** da scalare (`scaleTargetRef`): il Deployment su cui agire;
- I **limiti** di scaling: `minReplicaCount` e `maxReplicaCount`;

- I **trigger**: una lista di sorgenti di metriche con la soglia di attivazione. Quando almeno un trigger è attivo, KEDA crea l'HPA e avvia lo scaling.

Il numero di repliche viene calcolato, per ogni trigger, come:

$$\text{replicas} = \left\lceil \frac{\text{metricValue}}{\text{threshold}} \right\rceil \quad (2.2)$$

e il valore massimo tra tutti i trigger viene utilizzato come numero finale di repliche. Un trigger con metrica pari a zero viene considerato *inattivo*: se tutti i trigger sono inattivi e `minReplicaCount = 0`, KEDA può scalare il Deployment a zero repliche (*scale-to-zero*), funzionalità non disponibile nell'HPA nativo.

2.9 Prophet

Prophet è una libreria per la previsione di serie temporali sviluppata da Meta (Facebook) e pubblicata come open source nel 2017 da Taylor e Letham [6]. È progettata per essere robusta a dati mancanti, outlier e cambiamenti di trend.

Prophet modella la serie temporale come somma di quattro componenti:

$$y(t) = g(t) + s(t) + h(t) + \varepsilon_t \quad (2.3)$$

2.9.1 Fitting e overfitting

Il **fitting** è il processo di stima dei parametri del modello a partire dai dati di training. Prophet usa **ottimizzazione MAP** (*Maximum A Posteriori*) con il backend **Stan**: minimizza la funzione di perdita (verosimiglianza negativa più prior bayesiani) tramite l'algoritmo quasi-Newton **L-BFGS**, che converge in pochi secondi per serie di lunghezza moderata.

L'**overfitting** si verifica quando il modello diventa così flessibile da memorizzare il rumore del training set anziché il pattern sottostante: le previsioni risultano accurate sui dati noti ma inaccurate su quelli futuri. In Prophet il rischio è controllato da:

- `changepoint_prior_scale`: prior laplaciano sulle variazioni di pendenza del trend; valori alti permettono cambi frequenti (rischio overfitting), valori bassi lo rendono rigido;
- `fourier_order`: un ordine alto cattura stagionalità complesse ma può overfittare; per una sinusoide pura, $N = 4$ (default giornaliero) è sufficiente.

2.9.2 Trend $g(t)$

Cattura la crescita o il declino a lungo termine. Prophet supporta due modelli di trend: lineare a tratti (*piecewise linear*) e crescita logistica (*logistic growth*).

Crescita logistica (saturating growth). Adatto quando la serie tende verso un limite superiore C — la **capacità** — che rappresenta il massimo valore raggiungibile (es. massimo numero di utenti di un social network, capacità massima di server). La serie *satura* avvicinandosi asintoticamente a C senza mai superarla:

$$g(t) = \frac{C(t)}{1 + \exp\left(-(k + \mathbf{a}(t)^\top \boldsymbol{\delta})(t - (m + \mathbf{a}(t)^\top \boldsymbol{\gamma}))\right)} \quad (2.4)$$

dove $C(t)$ è la capacità (eventualmente variabile nel tempo), k è il tasso di crescita base, m è il parametro di offset e $\boldsymbol{\delta}$ sono le variazioni del tasso nei punti di cambiamento (*changepoints*).

Crescita lineare a tratti (piecewise linear). Più adatto a **serie senza saturazione**: il traffico web non ha un limite superiore intrinseco (può crescere senza bound), quindi imporre una capacità massima non ha senso fisico. Questo è il modello usato nel progetto:

$$g(t) = (k + \mathbf{a}(t)^\top \boldsymbol{\delta})t + (m + \mathbf{a}(t)^\top \boldsymbol{\gamma}) \quad (2.5)$$

dove $\mathbf{a}(t) \in \{0, 1\}^S$ è il vettore indicatore dei changepoint (i punti di cambiamento del trend) ($a_j(t) = 1$ se $t \geq s_j$), S è il numero di changepoint e $\boldsymbol{\gamma}$ sono le correzioni dell'intercetta necessarie per garantire la continuità della funzione nei changepoint:

$$\gamma_j = \left(s_j - m - \sum_{\ell < j} \gamma_\ell\right) \left(1 - \frac{k + \sum_{\ell < j} \delta_\ell}{k + \sum_{\ell \leq j} \delta_\ell}\right) \quad (2.6)$$

I changepoint vengono rilevati automaticamente da Prophet su una griglia uniforme di potenziali punti; il parametro `changepoint_prior_scale` controlla quanto aggressivamente il trend può cambiare pendenza, quindi la flessibilità del trend: valori alti permettono al trend di seguire i dati più da vicino (rischio overfitting), valori bassi lo rendono più rigido.

2.9.3 Stagionalità $s(t)$

Cattura variazioni periodiche ricorrenti. Prophet approssima la stagionalità tramite una **serie di Fourier** troncata:

$$s(t) = \sum_{n=1}^N \left[a_n \cos\left(\frac{2\pi nt}{P}\right) + b_n \sin\left(\frac{2\pi nt}{P}\right) \right] \quad (2.7)$$

dove P è il periodo della stagionalità (in giorni) e N è il `fourier_order`. I coefficienti a_n e b_n vengono stimati durante il fitting. Un `fourier_order` maggiore permette di catturare stagionalità più complesse e irregolari; un valore minore è sufficiente per forme più semplici come una sinusoide pura, ed è meno soggetto a overfitting.

Prophet supporta per default stagionalità giornaliera ($P = 1$, $N = 4$) e settimanale ($P = 7$, $N = 3$). Stagionalità con periodi diversi devono essere aggiunte esplicitamente con `add_seasonality()`.

2.9.4 Festività $h(t)$

Le festività modellano l'effetto di eventi speciali ricorrenti con date irregolari. Per ogni festività i si definisce l'insieme D_i delle date passate e future, e si costruisce il vettore indicatore $Z(t) \in \{0, 1\}^L$ (dove L è il numero totale di festività). Il contributo al modello è:

$$h(t) = \mathbf{Z}(t) \boldsymbol{\kappa}, \quad \kappa_i \sim \mathcal{N}(0, v_i^2) \quad (2.8)$$

dove v_i (controllato da `holidays_prior_scale`) regola l'ampiezza dell'effetto.

2.9.5 Termine di errore ε_t

Il termine di errore ε_t cattura le variazioni della serie che non sono spiegabili dalle componenti deterministiche $g(t)$, $s(t)$, $h(t)$. Taylor e Letham assumono $\varepsilon_t \stackrel{iid}{\sim} \mathcal{N}(0, \sigma^2)$, ovvero che il rumore residuo sia gaussiano, indipendente e identicamente distribuito.

2.9.6 Intervalli di incertezza

Prophet produce tre valori per ogni punto futuro: `yhat` (stima centrale MAP), `yhat_lower` e `yhat_upper` (limiti dell'intervallo di credibilità all'80% per default, stimati propagando l'incertezza sul trend futuro tramite simulazione). In questo progetto, per lo scaling preventivo, si usa `yhat_upper`: una sovrastima controllata garantisce risorse disponibili prima del picco.

Capitolo 3

Architettura del Sistema

3.1 Visione d'insieme

Il sistema è composto da cinque componenti principali deployati come pod Kubernetes, più un *Load Generator* usato esclusivamente per la simulazione. La figura 3.1 illustra i flussi di dati tra i componenti.

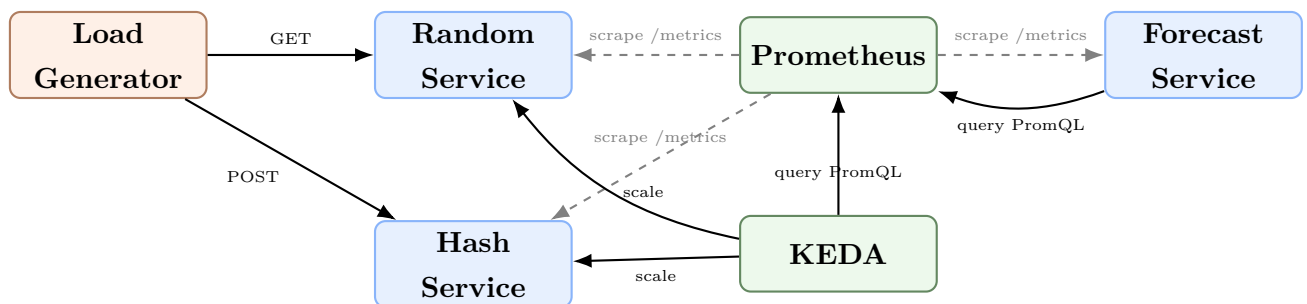


Figura 3.1: Architettura generale del sistema di predictive auto-scaling.

3.2 Componenti applicativi

Il sistema è composto da tre microservizi e uno strumento di simulazione, tutti containerizzati con Docker e orchestrati da Kubernetes.

I **microservizi applicativi** — Random Service e Hash Service — Ciascuno espone un'interfaccia web per l'interazione manuale e un endpoint `/metrics` per Prometheus. Sono i *target* dello scaling: il numero delle loro istanze viene adattato dinamicamente al carico previsto.

Il **Forecast Service** è il componente di intelligenza del sistema. Non gestisce traffico utente: osserva le metriche dei microservizi, costruisce il modello predittivo e pubblica le previsioni su Prometheus, fungendo da ponte tra l'osservabilità (Prometheus) e il motore di scaling (KEDA).

Il **Load Generator** è uno strumento di simulazione che genera richieste HTTP secondo un profilo sinusoidale configurabile, permettendo di osservare il comportamento del sistema predittivo in condizioni controllate.

3.3 Ciclo predittivo

Il ciclo predittivo si ripete ogni 5 minuti e coinvolge tutti i componenti:

1. Il Load Generator invia continuamente richieste HTTP ai microservizi secondo il profilo sinusoidale;
2. Prometheus raccoglie le metriche di traffico (`rate(requests[5m])`) ogni 30 secondi;
3. Il Forecast Service interroga Prometheus ogni 5 minuti, aggiunge il campione alla finestra storica, ri-addestra Prophet e pubblica la previsione per i prossimi 5 minuti come gauge Prometheus;
4. KEDA interroga Prometheus ogni 30 secondi, legge la previsione e aggiorna il numero di repliche dei microservizi di conseguenza.

3.4 Struttura del progetto

Directory	Contenuto
<code>random-service/</code>	Microservizio generatore di numeri casuali (porta 5000)
<code>hash-service/</code>	Microservizio calcolatore di hash SHA-256 (porta 5001)
<code>forecast-service/</code>	Servizio di previsione con Prophet (porta 8000)
<code>load-generator/</code>	Generatore di carico sinusoidale
<code>k8s/</code>	Manifest Kubernetes

3.5 Infrastruttura Kubernetes

3.5.1 Networking e Ingress

I servizi applicativi sono esposti internamente come `ClusterIP`: hanno un indirizzo IP stabile all'interno del cluster ma non sono raggiungibili dall'esterno. L'accesso esterno è gestito da un Ingress NGINX con routing basato su hostname:

- `random.local` → Random Service (porta 80 → 5000)
- `hash.local` → Hash Service (porta 80 → 5001)
- `prometheus.local` → Prometheus Server (porta 80)

3.5.2 Prometheus

Installato tramite il chart Helm `prometheus-community/prometheus`, Prometheus raccoglie le metriche di tutti i pod annotati con `prometheus.io/scrape: "true"` ogni 30 secondi. I dati vengono conservati nel database TSDB locale per un periodo di 15 giorni (default).

3.5.3 KEDA

Installato tramite il chart Helm `kedacore/keda` nel namespace `keda`, KEDA gestisce due `ScaledObject`: uno per Random Service e uno per Hash Service. Entrambi usano il trigger `prometheus` con la query `avg_over_time(predicted*_rate[5m])` e soglia `threshold = 1` req/s, con limiti `minReplicaCount = 1` e `maxReplicaCount = 5`.

Capitolo 4

Implementazione

4.1 Microservizi Flask

Entrambi i microservizi seguono la stessa struttura: un'applicazione Flask con due endpoint, uno per la logica applicativa e uno per le metriche Prometheus.

```
1 @app.route("/")
2 def random_number():
3     REQUEST_COUNT.inc()
4     value = random.randint(0, 1000000)
5     return render_template_string(HTML_PAGE, number=value)
6
7 @app.route("/metrics")
8 def metrics():
9     return generate_latest(), 200, {"Content-Type":
    CONTENT_TYPE_LATEST}
```

Listing 4.1: Random Service (random-service/app.py)

```
1 @app.route("/", methods=["GET", "POST"])
2 def hash_service():
3     hash_value = None
4     if request.method == "POST":
5         REQUEST_COUNT.inc()
6         text = request.form.get("input_text", "")
7         data = text.encode()
8         data = hashlib.sha256(data).digest()
9         hash_value = data.hex()
10
11     return render_template_string(HTML_PAGE, hash_value=hash_value)
```

Listing 4.2: Hash Service (hash-service/app.py)

Il Forecast Service utilizza Flask esclusivamente per esporre le metriche predette, poiché non gestisce traffico utente:

```

1 @app.route("/metrics")
2 def metrics():
3     return generate_latest(), 200, {"Content-Type":
    CONTENT_TYPE_LATEST}

```

Listing 4.3: Forecast Service – endpoint metriche

4.2 Forecast Service

4.2.1 Raccolta dati da Prometheus

Il Forecast Service interroga Prometheus tramite l'API HTTP `/api/v1/query`. Le query PromQL usate calcolano il rate di richieste per secondo nei 5 minuti precedenti: la finestra di 5 minuti è coerente con l'intervallo di campionamento del Forecast Service e garantisce una stima stabile del traffico corrente.

```

1 QUERY_RANDOM = "sum(rate(random_requests_total[5m]))"
2 QUERY_HASH   = "sum(rate(hash_requests_total[5m]))"
3
4 def query_prometheus(query):
5     r = requests.get(f"{PROM_URL}/api/v1/query", params={"query":
    query})
6     data = r.json()
7     if not data["data"]["result"]:
8         return 0.0
9     return float(data["data"]["result"][0]["value"][1])

```

Listing 4.4: Query a Prometheus (forecast-service/forecast.py)

4.2.2 Gestione della finestra storica

I campioni vengono accumulati in una lista con una dimensione massima di `WINDOW_SIZE = 2500` punti, corrispondente a circa 8 giorni di storia a un campione ogni 5 minuti. Mantenere una finestra finita impedisce che il modello sia influenzato da dati troppo vecchi e potenzialmente non più rappresentativi del pattern corrente.

```

1 WINDOW_SIZE = 2500
2 def add_point(history, value):
3     history.append({"ds": datetime.now(), "y": value})
4     if len(history) > WINDOW_SIZE:
5         history.pop(0)

```

Listing 4.5: Aggiunta di un campione alla finestra storica

4.2.3 Modello Prophet

Il traffico segue un ciclo giornaliero di 24 ore.

```
1 def train_model(history):
2     if len(history) < 10:
3         return None
4     df = pd.DataFrame(history)
5     model = Prophet()
6     model.fit(df)
7     return model
```

Listing 4.6: Addestramento del modello

4.2.4 Generazione della previsione

La previsione viene generata per un solo passo avanti (`FORECAST_STEPS = 1`), corrispondente a 5 minuti nel futuro. Questa scelta minimizza l'incertezza della previsione mantenendo comunque un anticipo sufficiente per KEDA ad avviare i nuovi pod prima del picco.

Se non esiste alcun modello allenato oppure lo storico delle metriche ha meno di 288 metriche (che corrisponde a 1 giorno di metriche), allora, al posto di una predizione, si passa l'ultima metrica presa. Così facendo, quando si hanno pochi dati per poter fare una previsione affidabile, si usa il *Reactive Scaling*.

```
1 FORECAST_STEPS = 1 # 5 minuti avanti
2
3 def forecast(model, history):
4     if model is None or len(history) < 288:
5         return history[-1]["y"]
6
7     future = model.make_future_dataframe(periods=FORECAST_STEPS,
8     freq="5min")
9     pred = model.predict(future)
10    return max(pred["yhat_upper"].iloc[-1], 0.0)
```

Listing 4.7: Generazione della previsione

4.2.5 Loop principale

Il loop principale si ripete ogni 300 secondi (5 minuti). Il tempo di esecuzione del training viene sottratto dallo sleep per mantenere il ritmo costante indipendentemente dalla durata del fitting di Prophet. In questo modo è assicurato che: la distanza tra le metriche prese siano almeno di 5 minuti; non vengano consumate troppe risorse non facendo allenare costantemente i modelli di Prophet.

```
1 def loop():
2     while True:
3         start_time = time.time()
4
5         add_point(history_random, query_prometheus(QUERY_RANDOM))
6         add_point(history_hash, query_prometheus(QUERY_HASH))
7
8         model_random = train_model(history_random)
9         model_hash = train_model(history_hash)
10
11        PRED_RANDOM.set(forecast(model_random, history_random))
12        PRED_HASH.set(forecast(model_hash, history_hash))
13
14        elapsed = time.time() - start_time
15
16        time.sleep(max(0, 300 - elapsed))
```

Listing 4.8: Loop principale del Forecast Service

4.3 Configurazione KEDA

La query `avg_over_time(...[5m])` calcola la media della metrica predetta negli ultimi 5 minuti: questo smorzamento filtra eventuali picchi istantanei della previsione, evitando oscillazioni rapide del numero di repliche.

Il trigger è considerato **attivo** quando il valore della query supera zero: KEDA crea allora l'HPA e mantiene il Deployment scalato secondo l'equazione (2.2). Quando la metrica scende a zero, il trigger diventa inattivo e l'HPA viene rimosso, lasciando il Deployment al valore `minReplicaCount`.

```
1 apiVersion: keda.sh/v1alpha1
2 kind: ScaledObject
3 metadata:
4   name: random-scaler
5 spec:
6   scaleTargetRef:
7     name: random-service
```

```

8   minReplicaCount: 1
9   maxReplicaCount: 5
10  triggers:
11  - type: prometheus
12    metadata:
13      serverAddress: http://prometheus-server.default.svc:80
14      metricName: predicted_random_rate
15      query: avg_over_time(predicted_random_rate[5m])
16      threshold: "1"

```

Listing 4.9: ScaledObject per il Random Service

4.4 Generatore di Carico

4.4.1 Motivazione del profilo sinusoidale

Il traffico web reale segue un *diurnal pattern*: picchi nelle ore diurne e minimi nelle ore notturne. Heller et al. [3] lo approssimano esplicitamente: “*To crudely approximate diurnal variation, we assume $u = \frac{1}{2}(1 + \sin(t))$, at time t , suitably scaled to repeat once per day*” (p. 8). Talluri et al. [4] adottano la stessa approssimazione: “*we assume that the number of users follows a diurnal pattern with peak usage in the evening; we approximate this with a shifted sine wave*” (p. 12).

4.4.2 Implementazione con Locust

Il generatore di carico è implementato con **Locust** [14], un framework open source per il load testing scritto in Python. A differenza di un semplice script che invia richieste in sequenza, Locust simula utenti virtuali concorrenti tramite coroutine *gevent*: ogni utente è una greenlet indipendente che esegue il proprio ciclo di richieste, permettendo di simulare migliaia di utenti concorrenti con risorse limitate.

Il profilo di carico è controllato dalla classe `LoadTestShape`, che definisce dinamicamente il numero di utenti virtuali in funzione del tempo. Il numero di repliche viene calcolato secondo:

$$\text{RPS}(t) = \frac{\text{MAX_RPS} + \text{MIN_RPS}}{2} + \frac{\text{MAX_RPS} - \text{MIN_RPS}}{2} \cdot \sin\left(\frac{2\pi t'}{T}\right) \quad (4.1)$$

dove $T = 86400$ s (24 ore) e t' è il tempo riallineato al picco delle ore 14:00 tramite `PEAK_HOUR` e `SHIFT_HOURS`.

Il numero di utenti virtuali (`user_count`) è proporzionale al RPS desiderato tramite `USERS_PER_RPS = 100`: ogni utente genera in media 1 req/s dato il `wait_time` tra 50

e 150 s (media 100 s $\rightarrow$ 1 req/100s per utente, quindi 100 utenti $\rightarrow$ 1 req/s effettivo). Il parametro `spawn_rate = user_count // 20` garantisce transizioni graduali (5% degli utenti per secondo), evitando variazioni brusche del carico.

```

1 import math, random
2 from datetime import datetime
3 from locust import HttpUser, task, between, LoadTestShape
4
5 HASH_HOST      = "http://hash-service.default.svc:80"
6 RANDOM_HOST    = "http://random-service.default.svc:80"
7 HASH_RATIO     = 0.5
8 MIN_RPS        = 1
9 MAX_RPS        = 10
10 PEAK_HOUR      = 14
11 SHIFT_HOURS    = 6
12 PERIOD_SEC     = 24 * 3600 # 24 ore
13 USERS_PER_RPS = 100
14
15 class WorkloadUser(HttpUser):
16     host = RANDOM_HOST
17
18     wait_time = between(USERS_PER_RPS * 0.5, USERS_PER_RPS * 1.5)
19
20     @task
21     def generate_load(self):
22         if random.random() < HASH_RATIO:
23             payload = {"input_text": f"load-test-{random.randint(0, 999999)}"}
24             self.client.post(HASH_HOST, data=payload,
25                             name="hash-service", timeout=10)
26         else:
27
28             self.client.get(RANDOM_HOST, name="random-service",
29                             timeout=10)
30
31 class SinusoidalLoad(LoadTestShape):
32     def tick(self):
33         now = datetime.now()
34         t = ((now.hour - (PEAK_HOUR - 12) - SHIFT_HOURS) % 24) *
35             3600 \
36             + now.minute * 60 + now.second
37
38         amplitude = (MAX_RPS - MIN_RPS) / 2
39         midpoint = (MAX_RPS + MIN_RPS) / 2
40
41         rps = midpoint + amplitude * math.sin(2 * math.pi * t /
42             PERIOD_SEC)

```

```
40     user_count = max(1, round(rps * USERS_PER_RPS))
41     spawn_rate = max(1, user_count // 20)
42     return (user_count, spawn_rate)
```

Listing 4.10: Generatore di carico Locust (load-generator/load_gen.py)

Il metodo `tick()` viene chiamato da Locust ogni secondo e restituisce la coppia `(user_count, spawn_rate)`: il numero totale di utenti virtuali desiderati e la velocità con cui aggiungerli o rimuoverli.

Capitolo 5

Risultati Sperimentali

5.1 Configurazione della simulazione

Parametro	Valore	Note
MIN_RPS	1 req/s	Valore minimo del ciclo
MAX_RPS	10 req/s	Valore massimo del ciclo
PERIOD_SECONDS	86400 s	Periodo giornaliero (24 ore)
HASH_RATIO	0.5	50% hash, 50% random

5.2 Profilo di traffico generato

Il traffico reale misurato da Prometheus mostra un andamento sinusoidale. I picchi si attestano intorno a 5,1 req/s, i minimi intorno a 0,4 req/s.

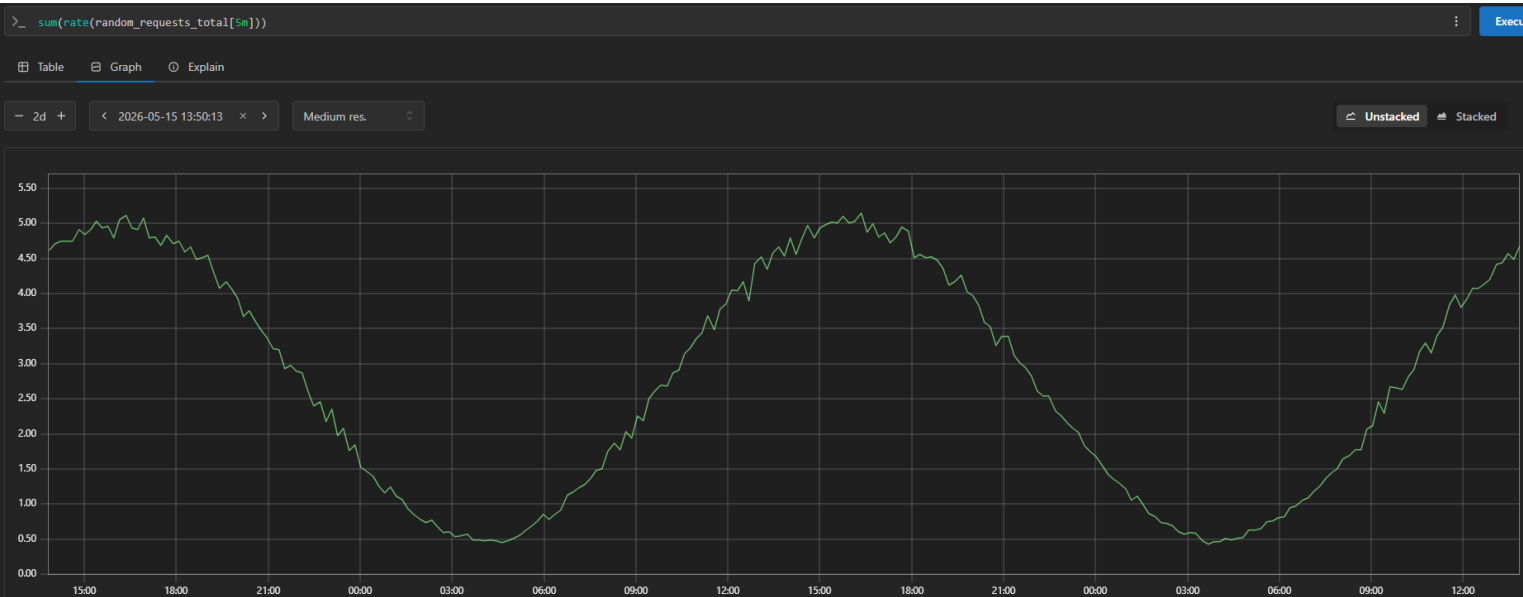


Figura 5.1: Traffico reale misurato con Prometheus al Random Service: `sum(rate(random_requests_total[5m]))`.

5.3 Metriche predette dal Forecast Service

5.3.1 Periodo di warm-up

Per il primo giorno ($288 \text{ campioni} \times 5 \text{ min}$) il Forecast Service restituisce l'ultimo valore misurato da Prometheus, l'adozione di questa misura fa sì che il Sistema riesca a funzionare correttamente, quindi a scalare i pod, anche affidandosi al *Reactive Auto-Scaling*. Questo impedisce scaling prematuri prima che il modello abbia dati sufficienti.

5.3.2 Fase predittiva

Dopo il warm-up, `predicted_random_rate` inizia ad oscillare in anticipo rispetto al traffico reale. I picchi si attestano intorno a 5,1 req/s, i minimi intorno a 0,6 req/s.

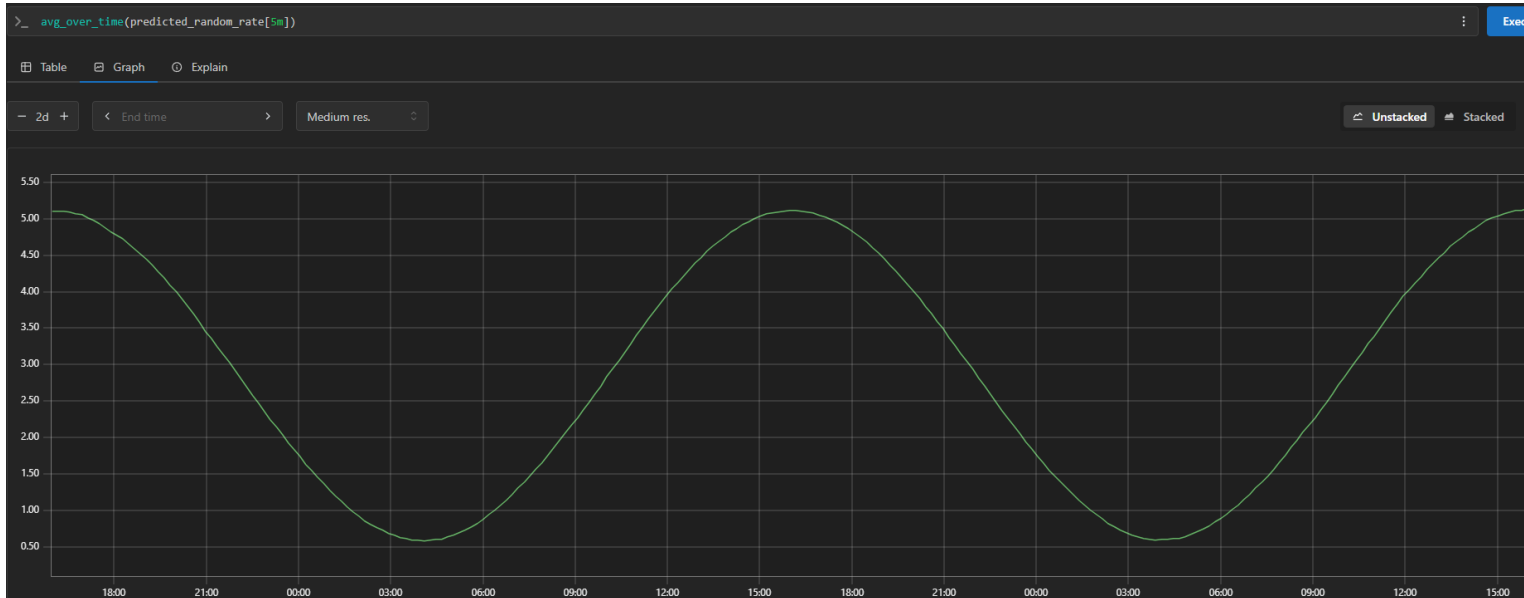


Figura 5.2: Metrica predetta `predicted_random_rate` esposta dal Forecast Service.

5.4 Comportamento dello scaling

5.4.1 Numero di repliche nel tempo

Fase	RPS predetto	Repliche	Anticipo sul picco
Minimo	0,6 req/s	1	—
Picco	5,1 req/s	5	5 min

Tabella 5.1: Comportamento dello scaling al variare del carico predetto.

5.4.2 Evidenza dello scaling

```

1 $ kubectl get pod | grep "random"
2 random-service-7f8d87885d-2txlv      1/1      Running    0      3h45m
3 random-service-7f8d87885d-fcstp      1/1      Running    0      3h13m
4 random-service-7f8d87885d-kl2n8      1/1      Running    0      2m2s
5 random-service-7f8d87885d-mv2xd      1/1      Running    0      7m3s
6 random-service-7f8d87885d-z64px      1/1      Running    0      10m

```

Figura 5.3: 5 repliche attive in prossimità del picco.

```

1 > kubectl describe scaledobject random-scaler
2 ...
3 Status:
4   Conditions:
5     Message: ScaledObject is defined correctly and is ready for scaling
6     Status: True   Type: Ready
7     Message: Scaling is performed because triggers are active
8     Status: True   Type: Active
9   Triggers Activity:
10     s0-prometheus:
11       Is Active: true

```

Figura 5.4: ScaledObject con trigger attivo.

5.5 Interfacce utente dei servizi



(a) Random Service (`random.local`)

(b) Hash Service (`hash.local`)

Figura 5.5: Interfacce web dei microservizi applicativi.

Capitolo 6

Conclusioni

6.1 Sintesi dei risultati

Il progetto ha dimostrato la fattibilità di un sistema di predictive auto-scaling completo, interamente basato su tecnologie open source, eseguito su un cluster Kubernetes locale. Il sistema implementato chiude un ciclo di feedback predittivo autonomo: raccolta di metriche (Prometheus), previsione del traffico (Prophet), scaling automatico (KEDA), senza alcun intervento manuale.

I risultati sperimentali mostrano che:

1. Il **Forecast Service** apprende il diurnal pattern del traffico dopo un warm-up di 1 giorno e genera previsioni stabili con un anticipo di 5 minuti rispetto al traffico corrente;
2. **KEDA** scala i Deployment in anticipo rispetto ai picchi reali, raggiungendo le 5 repliche massime;
3. L'uso di `yhat_upper` introduce un margine di sicurezza conservativo che garantisce disponibilità di risorse in tutte le fasi;
4. Il profilo sinusoidale, motivato dalla letteratura sul diurnal pattern [3, 4], si rivela particolarmente adatto per Prophet, che eccelle nel riconoscere stagionalità periodiche regolari.

6.2 Limiti del sistema

- **Warm-up:** Prophet richiede almeno 288 campioni (1 giorno) prima di produrre previsioni affidabili; durante questo periodo il sistema non scala preventivamente, ma in modo reattivo;

- **Ambiente single-node:** Minikube non è rappresentativo di un cluster di produzione in termini di latenze di rete e tempi di avvio dei pod;

6.3 Possibili evoluzioni

- **Multi-step forecasting:** aumentare `FORECAST_STEPS` per predire con maggiore anticipo (15–30 min), a costo di una riduzione di accuratezza;
- **Warm-starting:** riutilizzare i parametri del modello precedente (se presente) come punto di partenza del fitting, riducendo il tempo di computazione;
- **Modelli alternativi:** sperimentare con ARIMA, LSTM o Transformer per traffico più irregolare e non stazionario;
- **Deploy su cluster reale:** portare il sistema su GKE, EKS o AKS per validare il comportamento con traffico e risorse reali.

Bibliografia

- [1] P. Mell and T. Grance, *The NIST Definition of Cloud Computing*, NIST Special Publication 800-145, 2011. <https://doi.org/10.6028/NIST.SP.800-145>
- [2] The Linux Foundation, *Kubernetes — Production-Grade Container Orchestration*, <https://kubernetes.io/docs/concepts/overview/>
- [3] B. Heller, S. Seetharaman, P. Mahadevan, Y. Yiakoumis, P. Sharma, S. Banerjee, N. McKeown, *ElasticTree: Saving Energy in Data Center Networks*, NSDI '10, San Jose, CA, 2010. https://www.usenix.org/legacy/event/nsdi10/tech/full_papers/heller.pdf
- [4] S. Talluri, D. Niewenhuis, X. Chu, J. Kyselica, M. Cetin, A. Balgavy, A. Iosup, *Cloud Uptime Archive: Open-Access Availability Data of Web, Cloud, and Gaming Services*, arXiv:2504.09476, 2025. <https://arxiv.org/abs/2504.09476>
- [5] Prometheus Authors, *Prometheus — Monitoring System and Time Series Database*, <https://prometheus.io/docs/introduction/overview/>
- [6] S. J. Taylor and B. Letham, *Forecasting at Scale*, The American Statistician, 72(1):37–45, 2018. <https://doi.org/10.1080/00031305.2017.1380080>
- [7] KEDA Project, *Kubernetes Event-Driven Autoscaling — Official Documentation*, <https://keda.sh/docs/2.19/concepts/>
- [8] Docker Inc., *Docker — Accelerated Container Application Development*, <https://docs.docker.com/get-started/overview/>
- [9] Google LLC, *cAdvisor — Container Advisor*, <https://github.com/google/cadvisor>
- [10] Cloud Native Computing Foundation, *Container Network Interface Specification*, <https://github.com/containernetworking/cni>
- [11] Kubernetes, *minikube — Run Kubernetes Locally*, <https://minikube.sigs.k8s.io/docs/start/>

- [12] Pallets Projects, *Flask — Web Development, One Drop at a Time*, <https://flask.palletsprojects.com/>
- [13] P. J. Eby, *PEP 3333 – Python Web Server Gateway Interface v1.0.1*, Python Software Foundation, 2010. <https://peps.python.org/pep-3333/>
- [14] Locust Project, *Locust — An open source load testing tool*, <https://locust.io/>